SHREE GURU GOBIND SINGH TRICENTENARY UNIVERSITY

Gurugram, Haryana

Department of Computer Science & Engineering

TERM PAPER · THEMATIC ASSESSMENT

Database Management Systems Laboratory

Persistent Memory Architecture in Agents Using DBMS

Submitted by	**Harshit Khemani**
Roll number	2413020081
Programme	B.Tech, Computer Science & Engineering (AI/ML)
Section	C
Department	Computer Science & Engineering
Institution	SGT University

Academic Session 2026 · Semester V

Abstract

A language model does not remember anything. It reads a prompt and it writes a reply. After the reply, the model keeps nothing. An agent looks like it remembers because a separate program saves its history and sends that history back on the next call.

This paper studies that separate program. It shows that the program is a database, and that the design of agent memory is ordinary database design. The paper gives a four-layer architecture, a schema in PostgreSQL, an index plan, a recall query, and the rules that keep two agents from damaging each other's memory.

The paper then examines four real systems and asks one question of each: *how does this system remember something in session two that it learned in session one?* **OpenClaw** keeps facts in Markdown files and builds a SQLite index beside them. **Hermes Agent** keeps every message in one SQLite file and limits its curated notes by a hard size quota. **eve** saves the progress of the work itself, and **Benji**, built on eve at AgentsORG, copies a lesson from one project to all projects only after the lesson appears in three projects. **Poke** keeps a separate log for each worker agent, and **Code Storage** stores work as Git objects, where a merge settles the conflict that a plain database can only reject.

KEYWORDS

agent memory, database management systems, sessions, schema design, normalisation, indexing, recall, SQLite, PostgreSQL, Git.

A CORRECTION TO TWO COMMON CLAIMS

OpenAI did not buy OpenClaw. OpenAI hired the person who created it in February 2026. The project moved to an independent foundation and it kept the MIT licence [14, 15]. Cognition bought The Interaction Company, which makes Poke, in July 2026 [26]. No public record shows that anybody bought the Pierre Computer Company [29]. Section 7.4 marks clearly which statements come from the documents and which statements are my own design.

Contents

List of Figures

List of Tables

1 Introduction

1.1 An agent forgets when a session ends

A session is one continuous conversation between a user and an agent. Inside a session the agent looks like it remembers. The program simply resends the earlier text on every call.

The illusion breaks at the session boundary. The program starts the next session with an empty prompt. Everything the agent learned on Monday is gone on Thursday, unless some other component saved it.

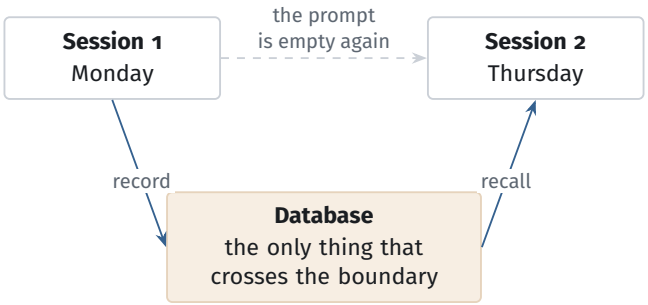

Figure 1. Nothing survives a session boundary by itself. The agent must record what it learns, and it must recall that material at the start of the next session.

Two words carry a fixed meaning through this paper. To **record** means to write something into the database. To **recall** means to read something back out of it and put it into the prompt.

1.2 Why a file is not enough

Many agents start with a text file. A file works until the agent grows. Then five problems appear, and every one of them is a problem that databases already solve.

1. One new fact touches several records. All of them must land, or none of them.
2. The agent can stop in the middle of a task. It must not lose work that it already reported as finished.
3. Two helper agents can write at the same time. A plain file loses one of the two writes.
4. The agent must find facts by meaning and by exact word, and it must filter by date and by owner.
5. Facts expire. A file keeps the old fact and the new fact side by side, and the agent then reads both.

1.3 Scope and method

This paper covers how an agent keeps memory across sessions, and how a database supports that. It does not cover model training, prompt design, or agent planning.

I built the four case studies from public documents, public source repositories, and reports. I cite each source where I use it. I ran no experiments, so this paper reports no measurements of its own. Where a company states a speed figure, I mark it as the company's claim.

2 What the Agent Must Keep

An agent produces four kinds of material. Each kind has a different size, a different read pattern, and a different cost when it is lost. One storage design cannot serve all four well.

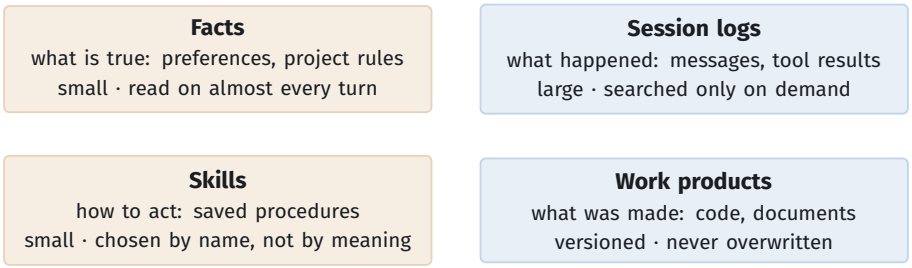

Figure 2. The four kinds of material an agent must keep. The two on the left stay small and the agent loads them often. The two on the right grow without limit and the agent opens them only when it needs them.

Table 1. How each kind of material maps onto database structures.

Kind	Database structure	Lifetime
Facts	A small table of rows. Each row carries a subject, a statement, a source, and a validity period.	Until a newer fact replaces it
Session logs	A large append-only table with a full-text index.	Kept, then moved to cheap storage
Skills	A catalogue table with a name, tags, and a use count.	Until somebody revises it
Work products	Content-addressed objects such as Git commits.	Permanent and versioned

The four kinds also differ in what a loss costs. If the agent loses a session log, its recall gets weaker. If it loses a fact, its behaviour changes. If it loses a work

product, real work disappears. The system should therefore protect the three kinds by different amounts.

3 What a Database Gives the Agent

Table 2 lists what agent memory needs and names the database mechanism that supplies it. The right column is the argument of this paper in short form. Database systems already supply every item.

Table 2. Requirements for agent memory, and the mechanism that meets each one.

Requirement	Mechanism
Work must survive a crash	Write-ahead log and checkpoints
One fact writes several rows together	Transactions
Two agents write at once	Multiversion concurrency control
A recalled fact must point at a real message	Foreign keys
Search by meaning and by exact word	A vector index and a text index
Old facts must stop appearing	A validity period on every fact
An operator must see why the agent believes something	A link to the source message
The prompt must stay small	A strict limit on how many rows recall returns
One system serves many users	A tenant column and row-level security

Two of these rows deserve a note.

The prompt limit is the hardest constraint. A normal database tries to read as few disk pages as possible. Here the scarce resource is the prompt. A recall that returns fifty correct rows has failed if six rows were enough. The memory layer must therefore throw away most of what it finds.

The validity period is the row that deployed systems most often skip. Without it the agent recalls a fact and its replacement at the same time. Section 5.5 shows the repair.

4 The Architecture

I separate the design into four layers. Each layer holds a different kind of material and offers a different guarantee.

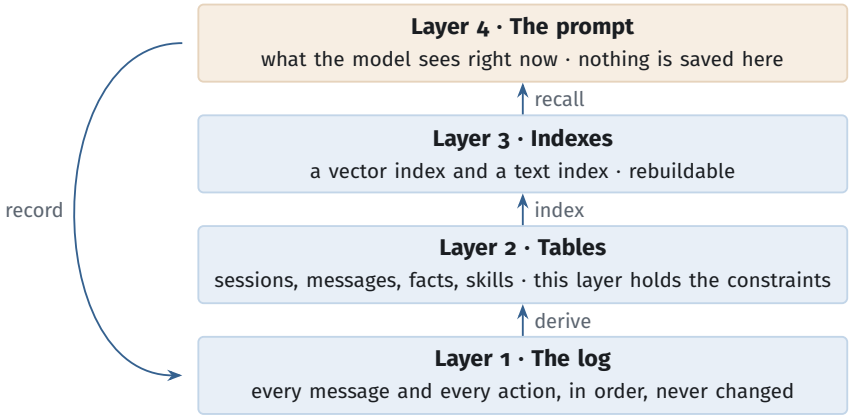

Figure 3. The four layers. Layer 1 is the only original copy. The system can rebuild layers 2 and 3 from it. Layer 4 disappears after every model call.

Layer 1 holds the log. The agent appends every message, every tool call, and every result. It never edits a row here. This layer buys three things. The team can drop layers 2 and 3 and build them again when the embedding model changes. An operator can replay the log and see why the agent acted. A crashed agent can restart from the last entry.

Layer 2 holds the tables. This layer stores sessions, messages, facts, and skills. Constraints live here, so this is the layer that makes promises. Section 5 gives the schema.

Layer 3 holds the indexes. An index is derived data. It can fall behind the table, and the team can always build it again.

Layer 4 is the prompt. It is small, expensive, and temporary. Every other layer exists to keep it short and correct.

4.1 How the agent records a fact

Recording is not one `INSERT`. The agent runs five steps, and the first four must succeed together or fail together.

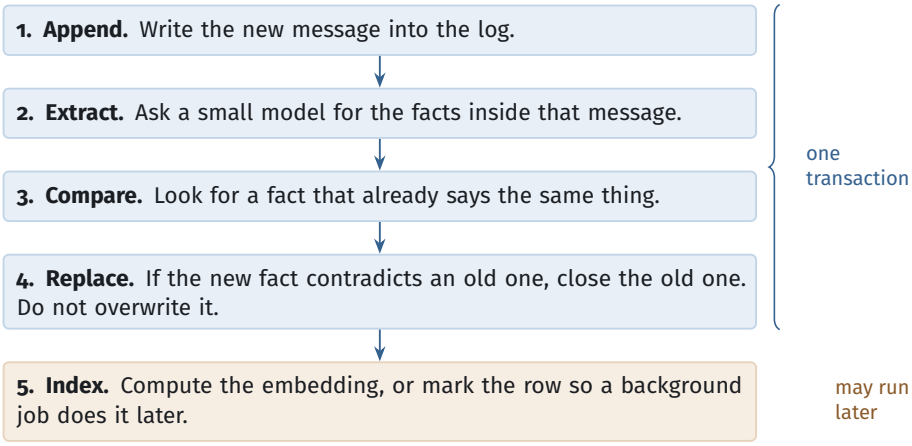

Figure 4. The record path. Step 5 can run later, which is exactly why the schema carries a flag that marks an index entry as out of date.

4.2 How the agent recalls a fact

At the start of a session the agent asks the database for a small set of rows. Two searches run, and the system merges their results.

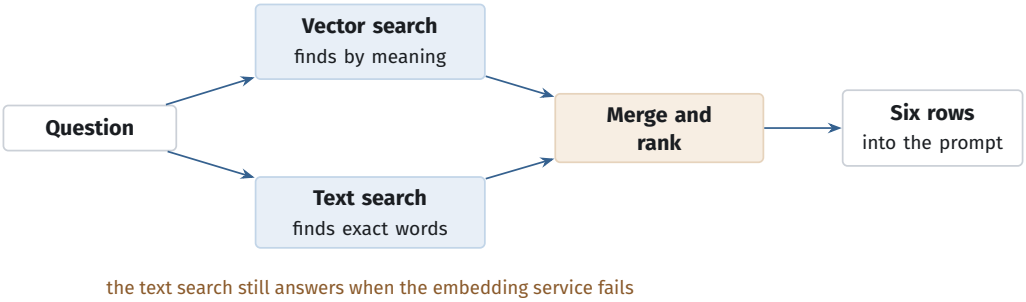

Figure 5. The recall path. Vector search alone misses file names, error strings, and version numbers, so every system in this paper runs a text search beside it.

The system ranks the merged rows by three signals. It prefers a row that both searches found. It prefers a row that the user marked as important. It prefers a recent row, unless somebody marked the row as permanent. OpenClaw halves the weight of a dated note every thirty days and never reduces the weight of a curated note [13].

4.3 How the agent forgets

Three different operations hide behind the word *forget*. Keep them separate.

- **Rank lower.** The row stays. Recall stops returning it.

- **Move.** The row goes to cheap storage. The agent can still read it, but more slowly.
- **Delete.** The row goes away for good. This is a legal action, not a speed improvement, and the team must also redact the log entry.

A fourth method works better than all three for curated notes. Hermes Agent sets a hard size limit and returns an *error* when a write would pass it [18]. The agent must then merge or drop a note before it can add one. A quota is easier to reason about than a background cleanup job.

5 The Schema

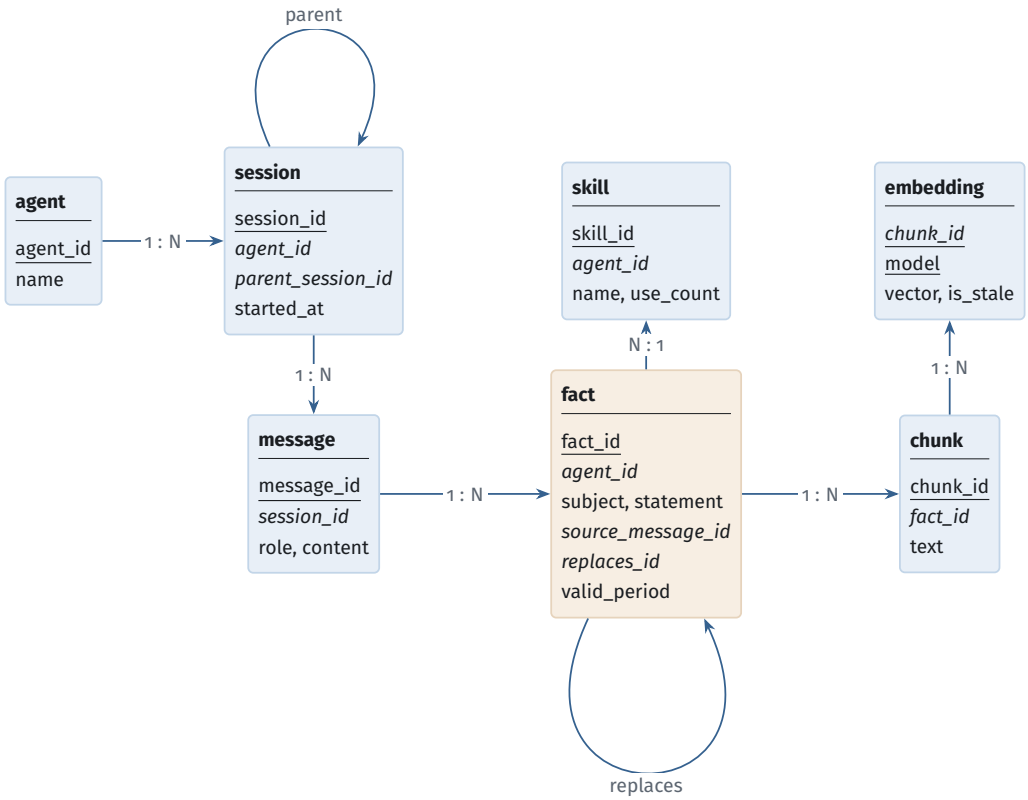

Figure 6. The schema. Two relationships point back at their own table. `parent_session_id` keeps the link when the agent shortens a long session, and `replaces_id` lets a new fact retire an old one without erasing it. Underline marks a primary key. Italics mark a foreign key.

5.1 The tables

```
CREATE TABLE session (
    session_id        uuid PRIMARY KEY,
    agent_id          uuid NOT NULL REFERENCES agent ON DELETE CASCADE,
```

```
    -- when the agent shortens a session, the short one points here
    parent_session_id uuid REFERENCES session(session_id),
    channel           text NOT NULL
                        CHECK (channel IN ('cli','web','chat','api')),
    started_at        timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE message (
    message_id  bigserial PRIMARY KEY,
    session_id  uuid NOT NULL REFERENCES session ON DELETE CASCADE,
    seq         integer NOT NULL,
    role        text NOT NULL
                  CHECK (role IN ('user','assistant','tool')),
    content     text NOT NULL,
    content_tsv tsvector GENERATED ALWAYS AS
                  (to_tsvector('english', content)) STORED,
    UNIQUE (session_id, seq)
);

CREATE TABLE fact (
    fact_id           bigserial PRIMARY KEY,
    agent_id          uuid NOT NULL REFERENCES agent ON DELETE CASCADE,
    subject           text NOT NULL,
    statement         text NOT NULL,
    is_permanent      boolean NOT NULL DEFAULT false,
    -- why does the agent believe this?
    source_message_id bigint REFERENCES message ON DELETE SET NULL,
    replaces_id       bigint REFERENCES fact(fact_id),
    -- the period during which this fact is true
    valid_period      tstzrange NOT NULL DEFAULT tstzrange(now(), NULL),
    last_used_at      timestamptz NOT NULL DEFAULT now(),
    -- the agent holds ONE live fact per subject at any moment
    EXCLUDE USING gist (agent_id WITH =, subject WITH =,
                        valid_period WITH &&)
);

CREATE TABLE embedding (
    chunk_id  bigint NOT NULL REFERENCES chunk ON DELETE CASCADE,
    model     text NOT NULL,
    vector    vector(1536) NOT NULL,
    is_stale  boolean NOT NULL DEFAULT false,
    PRIMARY KEY (chunk_id, model)
);
```

Listing 1. The core tables in PostgreSQL.

5.2 Normalisation

A first version of an agent usually writes one wide table. It stores the session, the agent name, the model name, the message, the extracted fact, and the skill name in a single row. That table repeats the agent name on every row, and it repeats the model name with it. Table 3 shows what breaks.

Table 3. Three faults in the single wide table, and what each one costs.

Fault	What happens
Update fault	An operator changes the agent's model. The database must rewrite every historical row.
Insert fault	A session starts with no messages yet. The table cannot record the session at all.
Delete fault	Somebody deletes the last message of a session. The fact that the session existed disappears with it.

The repair splits the wide table into `agent`, `session`, `message`, `fact`, `chunk`, `embedding`, and `skill`. Each table then has one key, and every column describes that key and nothing else. The schema reaches Boyce–Codd normal form. Every join follows a foreign key, so the split loses no information.

I break the rule in three places on purpose:

- `tool_calls` stays as one JSON column. Its shape changes with every model provider, so a fixed set of columns would be wrong within a year.
- `agent_id` repeats on child tables. A security rule then reads it without a join, and that rule runs on every query.
- `content_tsv` repeats the text in searchable form. This buys the text index, and it is the same trade any index makes.

5.3 Indexes

Table 4. One index for each kind of question the agent asks.

Question	Index	Reason
"What is similar?"	HNSW on the vector	Finds near vectors without reading them all
"Which row has this word?"	GIN on the text	Finds names and error strings that vectors miss
"Which rows are live now?"	GiST on the period	Also enforces the one-live-fact rule
"Which rows belong to this agent?"	Partial B-tree	Keeps retired rows off the fast path

```
CREATE INDEX ix_emb_vec  ON embedding USING hnsw (vector vector_cosine_ops);
CREATE INDEX ix_fact_tsv ON fact USING gin  (to_tsvector('english', statement));
CREATE INDEX ix_fact_per ON fact USING gist (valid_period);

-- Recall only ever reads facts that are still true.
```

```
CREATE INDEX ix_fact_live ON fact (agent_id, last_used_at DESC)
    WHERE upper_inf(valid_period);
```

Listing 2. The indexes.

5.4 The recall query

```
WITH live AS (
    -- Apply the owner filter and the date filter one time only.
    SELECT f.fact_id, f.statement, f.is_permanent, f.last_used_at
    FROM   fact f
    WHERE  f.agent_id = $1
      AND  f.valid_period @> now()
),
by_meaning AS (
    SELECT l.fact_id,
           ROW_NUMBER() OVER (ORDER BY MIN(e.vector <=> $2)) AS place
    FROM   live l
      JOIN chunk     c ON c.fact_id  = l.fact_id
      JOIN embedding e ON e.chunk_id = c.chunk_id AND NOT e.is_stale
    GROUP BY l.fact_id
    LIMIT 200
),
by_word AS (
    SELECT l.fact_id,
           ROW_NUMBER() OVER (
             ORDER BY ts_rank_cd(to_tsvector('english', l.statement),
                                 websearch_to_tsquery('english', $3)) DESC
           ) AS place
    FROM   live l
    WHERE  to_tsvector('english', l.statement)
             @@ websearch_to_tsquery('english', $3)
    LIMIT 200
)
SELECT l.fact_id, l.statement
FROM   live l
  LEFT JOIN by_meaning m ON m.fact_id = l.fact_id
  LEFT JOIN by_word    w ON w.fact_id = l.fact_id
WHERE  m.place IS NOT NULL OR w.place IS NOT NULL
ORDER  BY COALESCE(m.place, 999) + COALESCE(w.place, 999),
          l.is_permanent DESC,
          l.last_used_at DESC
LIMIT  6;    -- six rows, because the prompt is the scarce resource
```

Listing 3. Recall. The two searches run, the results merge, and the query returns six rows.

5.5 Old facts must not come back

Every fact carries a period. When the agent learns something that contradicts an old fact, it closes the old period and opens a new one. It does not delete the old row.

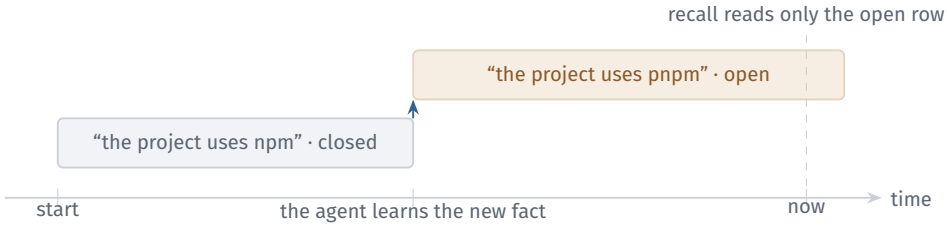

Figure 7. A new fact retires an old one. The old row stays for the audit trail, but recall never returns it again.

This design also answers a question that a plain table cannot answer: *what did the agent believe last month?* An operator changes one parameter in the query and reads the state at any past date.

6 Two Agents That Write at the Same Time

Modern agents start helper agents. When two helpers write to the same memory, the system meets the classic problems of any multi-user database. Table 5 lists the three that matter here.

Table 5. Three faults that appear when two agents write together.

Fault	What happens	Repair
A write disappears	Both helpers read the notes file, both add a line, and the second write replaces the first.	Write single rows, not whole files
A fact appears twice	Both helpers look for a fact, both find nothing, and both insert it.	The `EXCLUDE` rule in the schema
An action repeats	The agent sends an email, then crashes before it saves its progress. On restart it sends the email again.	A table of completed actions

The second fault is the one teams miss. The two helpers write *different* rows, so the database sees no conflict between them. Each helper simply read a set that the other helper then changed. A common isolation setting allows this. The schema in Section 5 closes it, because the `EXCLUDE` rule makes two live facts about one subject impossible to store. A rule in the schema holds whatever the isolation setting is.

The third fault needs one small table. The agent records the action and performs it inside the same transaction.

```
INSERT INTO completed_action (action_key, session_id, kind)
VALUES ($1, $2, 'send_email')
ON CONFLICT (action_key) DO NOTHING
```

```
RETURNING action_key;   -- no row means the agent already did it
```

Listing 4. A table of completed actions stops a repeat after a crash.

7 How Four Systems Remember

This section asks one question of four real systems. *How does the system remember in session two what it learned in session one?*

7.1 OpenClaw

OpenClaw is an open assistant that runs on the user's own machine. A local gateway manages its sessions, its channels, and its tools [12].

How it remembers. OpenClaw writes its long-term facts into Markdown files. `MEMORY.md` holds facts about the user. `USER.md` holds the user's profile. A dated file under `memory/` holds each day's notes. The system then reads those files, splits them into pieces, and stores the pieces with their vectors in a SQLite database [13]. At the start of a new session the agent searches that database and loads at most six pieces back into the prompt.

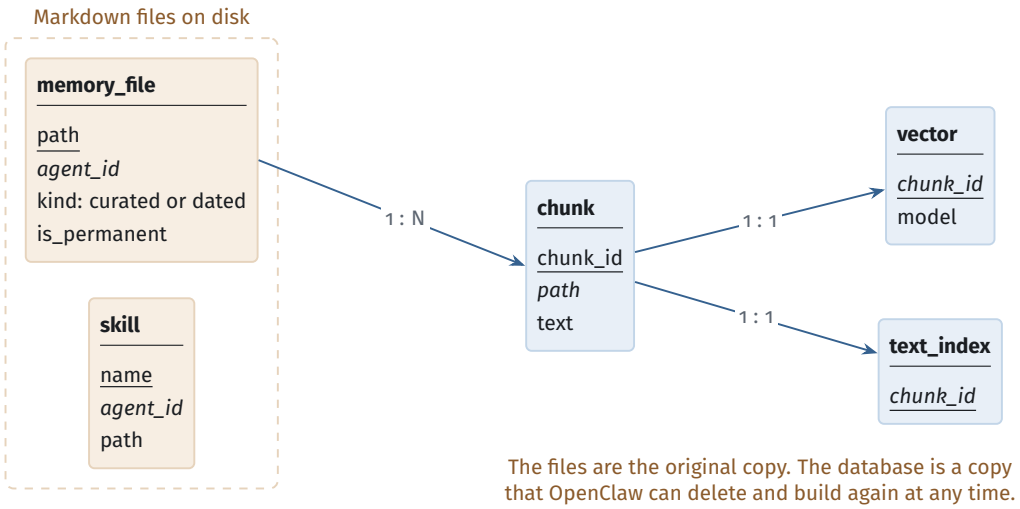

Figure 8. OpenClaw. Clay boxes are Markdown files and blue boxes are SQLite tables. Underline marks a primary key. Italics mark a foreign key.

What this design buys. The files stay readable, so a person can open them and correct a wrong fact. The database is a copy, so OpenClaw can delete it and build it again when the embedding model changes. The documents state that the team must run a rebuild command after such a change [13]. Text search also acts as the safety net. When the embedding service fails, OpenClaw still answers from the text index alone.

What it costs. OpenClaw keeps one database file for each agent. A query that spans two agents therefore cannot use a join. The program must read both files and merge the results itself.

One note on the project. OpenAI hired the creator of OpenClaw in February 2026. OpenAI did not buy the project. The project moved to an independent foundation and it kept the MIT licence [14, 15].

7.2 Hermes Agent

Hermes Agent comes from Nous Research. It runs from a terminal, from a chat gateway, and from a code editor, and all three paths use one agent class [17].

How it remembers. Hermes splits memory in two. Every message from every session goes into one SQLite file. A full-text index sits over those messages, so the agent can search months of history and read the real sentences back [18]. Beside that file sit two small Markdown notes. `MEMORY.md` holds about 800 tokens and `USER.md` holds about 500 tokens. Hermes loads both notes at the start of a session and fixes them into the prompt.

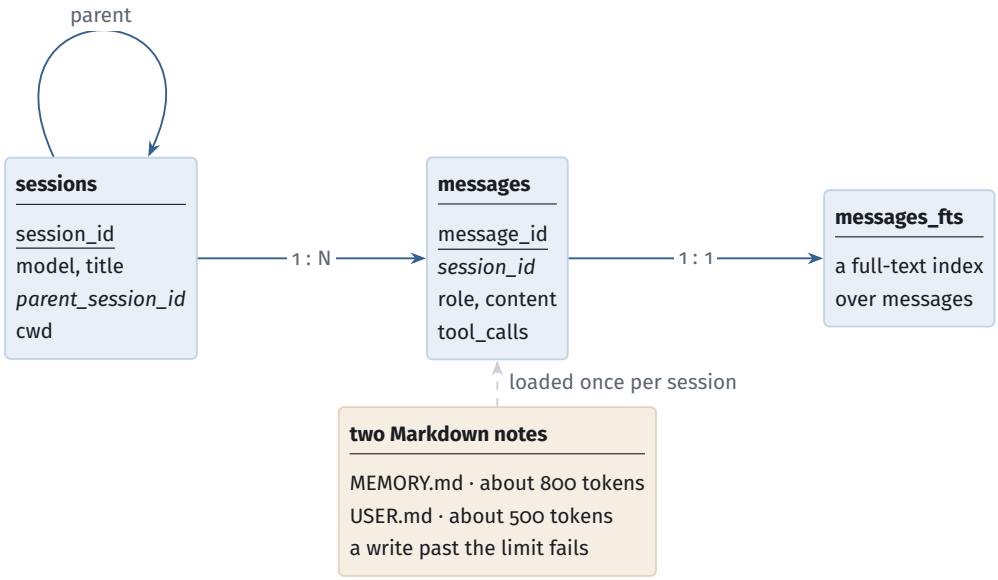

Figure 9. Hermes Agent. One SQLite file holds unlimited history. Two small files hold the notes, and those notes have a hard size limit. Underline marks a primary key. Italics mark a foreign key.

Three decisions worth copying. First, the notes have a hard limit and a write past that limit returns an error [18]. The agent must therefore merge two notes before it writes a third. A quota beats a cleanup job, because the agent sees the quota and can act on it. Second, a session that grows too long becomes a shorter child session, and `parent_session_id` keeps the link [19]. Shortening a session therefore adds a

row instead of destroying one. Third, Hermes opens the database in write-ahead logging mode so readers do not block the writer, and it falls back to a simpler mode on network drives that cannot support it [19, 30].

The trade. Hermes fixes the notes into the prompt at session start. A note that the agent writes during a session does not appear until the next session [18]. Hermes accepts a stale read in exchange for a cheaper prompt.

7.3 eve and Benji

eve is Vercel's framework for agents that survive a restart. An agent is a folder [20]. Benji is a design-review agent that AgentsORG builds on eve [23, 24].

How eve remembers. eve saves the progress of the work, not the meaning of it. It splits work into sessions, turns, and steps, and it writes a checkpoint at every step boundary [21]. When the process restarts, it continues from the last finished step instead of repeating the whole turn. A durable store holds this state, and the team chooses that store. On a server it can be PostgreSQL.

```
export default defineAgent({
  model: "anthropic/claude-opus-4.8",
  experimental: { workflow: { world: "@workflow/world-postgres" } },
});
```

Listing 5. Choosing PostgreSQL as the durable store in eve [21].

eve also offers a named slot of durable state for one session. The documents are strict about its limit. The slot belongs to one session, a helper agent never shares it, and anything that must outlive the session belongs in a separate database [22]. That limit is the reason two helper agents under eve cannot damage each other's memory. They hold no shared cell.

How Benji remembers. Benji keeps two levels. The first level holds notes about one repository, such as a person's stated preference. The second level holds portable lessons. A note becomes a lesson only after it appears in three separate repositories [23].

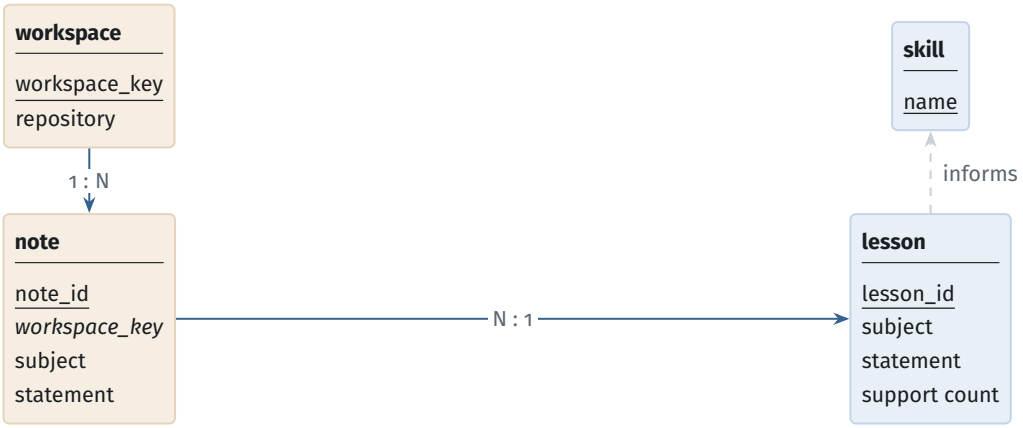

Figure 10. Benji. The rule that copies a note into a lesson is a count with a threshold, and one SQL statement expresses it. Underline marks a primary key. Italics mark a foreign key.

The rule stops one project's habit from becoming a rule for every project. It is a counting rule, and SQL states counting rules directly.

```
INSERT INTO lesson (agent_id, subject, statement, support_count)
SELECT n.agent_id, n.subject,
       (ARRAY_AGG(n.statement ORDER BY n.last_used_at DESC))[1],
       COUNT(DISTINCT n.workspace_key)
FROM   note n
GROUP  BY n.agent_id, n.subject
HAVING COUNT(DISTINCT n.workspace_key) >= 3;
```

Listing 6. Benji's promotion rule. The last line is the whole policy.

Where Benji stands today. Benji stores both levels in JSON files [23]. That choice keeps the data readable, and it costs the three things this paper has listed. JSON files offer no transaction, no uniqueness rule, and no index, so the promotion rule above must run as program code over every note on every start. *The following plan is my own design work for AgentsORG. Benji does not ship it today.* Move both levels into the `fact` table of Section 5, run the promotion rule on a schedule, and keep a JSON export so a person can still read the agent's beliefs.

7.4 Poke and Code Storage

Poke is an assistant that lives inside a messaging app. It started in March 2026 and it carried more than 100 million messages in three months. Cognition bought the company in July 2026 [26].

How it remembers. Poke separates the agent that talks from the agents that work [25]. One interaction agent holds the conversation. It starts a worker agent for each task, and each worker keeps its own complete log of every action and every

result. A worker survives its task, so a question next month reaches the same worker with its log intact. Poke also treats the user's mailbox as a third store that it reads but never writes.

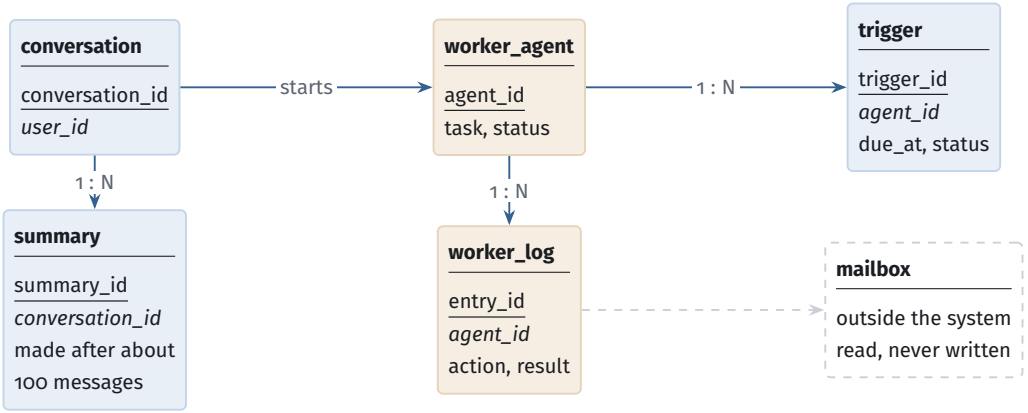

Figure 11. Poke. Each worker owns its own log, so two workers never write to one place. Underline marks a primary key. Italics mark a foreign key.

Poke keeps its reminders in a SQL table and checks that table once a minute [25]. A worker may change only its own reminders. A poll every minute is slower than a timer, but it restarts cleanly after a crash, and a simple query expresses it.

Code Storage, and Git as a database. The Pierre Computer Company sells Git storage for machines. It offers commits, branches, and merges through an API, and it adds short-lived branches and writes that need no local copy [27, 28]. Git belongs in this paper because Git is a database. Table 6 translates it.

Table 6. Git parts, read as database parts.

Git	Database
Object store	Content-addressed rows. Two identical files share one row automatically.
Commit	One complete version of the data, with a pointer to the version before it.
Branch	A snapshot that one writer works on alone.
Branch update	A compare-and-swap. This is why a push fails instead of overwriting.
Merge	The step that settles two changes. A plain database can only reject one of them.
Reflog	The write-ahead log and the audit trail.

The merge row matters most. When two agents change the same thing, a key-value store keeps the last write and loses the other one. A relational database stops one

of the two transactions. Git merges them, and it asks a person only when it cannot. Agent memory needs that third answer, because two agents are often both right.

The next paragraph is my own design, not a description of a product. No public source says that Poke uses Code Storage. I pair them because Poke's per-task workers and Code Storage's short-lived branches fit together. Give each worker its own branch. Starting the branch is the start of a transaction. Each commit is a private write. Merging the branch is the commit, and the branch update fails if the main line moved, which is exactly an optimistic retry. Dropping an abandoned branch costs nothing.

8 Comparison

Table 7. How the four systems carry memory across a session boundary.

System	What crosses the boundary	The idea worth copying
OpenClaw	Markdown files, with a SQLite index built from them	Keep the original readable and treat the index as a copy
Hermes	One SQLite file of every message, plus two size-limited notes	Set a quota and fail the write, instead of cleaning up later
eve and Benji	Step checkpoints in a durable store, plus Benji's two levels of notes	Give each helper its own state, so no two can collide
Poke and Code Storage	A private log for each worker, plus reminders in a SQL table	Use a merge to settle a conflict, not a rejection

8.1 What the four systems agree on

They all use more than one store. Every system keeps small facts apart from large history. The two have different sizes and different read patterns, so one engine cannot serve both well.

They all keep files. Three of the four keep the curated facts in readable files and use the database as the index. A person can open a file, read what the agent believes, and correct it. This matters more than it first appears.

They all search twice. No system searches by meaning alone. Vectors miss file names, error strings, and version numbers, so a text search always runs beside the vector search.

They all set limits. Hermes sets a token quota. OpenClaw returns six rows. Benji needs three repositories. A store that grows without a limit gets slower and less accurate at the same time.

They avoid sharing instead of locking. None of the four uses a strict isolation level to coordinate writers. eve gives each helper its own state. Poke gives each worker its own log. Git gives each agent its own branch. They remove the shared cell rather than guard it. This works until the agents must share what they learned, which is exactly Benji's promotion step.

They all skip validity periods. No system in this study records when a fact stops being true. Databases solved this problem long ago. Agents have not adopted it yet.

9 Design Rules

1. **Keep the log as the original.** Build every index from it. A wrong index then costs a rebuild, not the data.
2. **Return few rows.** Six correct rows beat fifty correct rows, because the prompt is the scarce resource.
3. **Set a quota and fail the write.** The agent then sees the limit and merges its notes itself.
4. **Give each helper its own state.** Add a constraint only at the few points where helpers must share.
5. **Put the safety rule in the schema.** A constraint holds even when the program is wrong.
6. **Give every fact a period.** Close the old period, open a new one, and never overwrite.
7. **Keep the memory readable.** A person must be able to open it and correct it.

10 Limits of This Study

I ran no experiments. The schema and the queries compile, but I did not measure them against a workload, so every performance statement here is reasoning and not a result. I read public documents and public source, so the descriptions can lag the code. Poke is closed software, and I built that section from a third-party write-up [25]. All four projects change quickly, so a reader should check the current documents before relying on a table name or a setting. I chose the four systems because they are well documented and different from each other, not by any sampling rule.

11 Conclusion

An agent's memory is not part of the model. It is a database, and the agent behaves well over weeks only when that database is well designed.

The four layers in this paper separate a log that never changes from tables that carry constraints, an index that the team can rebuild, and a prompt that the system discards after every call. Each separation follows from a difference in access pattern or in the cost of a loss. The schema that results is ordinary. It normalises to Boyce–Codd normal form, it takes ordinary indexes, and one SQL statement expresses the recall.

The four systems resolve the same tensions in four defensible ways. OpenClaw shows that a readable original plus a rebuildable index is a strong pairing. Hermes shows that a hard quota produces a more predictable agent than unlimited growth. eve shows that a framework should refuse to widen the scope of its state, and Benji shows that a counting rule stops one project's habit from spreading. Poke shows that one log per worker removes a whole class of concurrency fault, and Code Storage shows that Git already offers the merge that relational memory lacks.

Two gaps remain. No system records when a fact stops being true. No agreed method exists for two agents that must write to one memory, only the practice of avoiding shared state. Both gaps have answers in the database literature. The field would move faster if it read them.

References

[1] E. F. Codd. "A Relational Model of Data for Large Shared Data Banks." *Communications of the ACM*, 13(6):377–387, 1970.

[2] T. Härder and A. Reuter. "Principles of Transaction-Oriented Database Recovery." *ACM Computing Surveys*, 15(4):287–317, 1983.

[3] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil and P. O'Neil. "A Critique of ANSI SQL Isolation Levels." *Proceedings of ACM SIGMOD*, 1995.

[4] A. Fekete, D. Liarokapis, E. O'Neil, P. O'Neil and D. Shasha. "Making Snapshot Isolation Serializable." *ACM Transactions on Database Systems*, 30(2):492–528, 2005.

[5] R. T. Snodgrass. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann, 2000.

[6] M. Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[7] Y. A. Malkov and D. A. Yashunin. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs." *IEEE TPAMI*, 42(4):824–836, 2020.

[8] P. Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.” *NeurIPS*, 2020.

[9] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang and M. S. Bernstein. “Generative Agents: Interactive Simulacra of Human Behavior.” *Proceedings of ACM UIST*, 2023.

[10] C. Packer et al. “MemGPT: Towards LLMs as Operating Systems.” arXiv:2310.08560, 2023.

[11] J. Gruber and J.-N. Hilgert. “Foundations for Agentic AI Investigations from the Forensic Analysis of OpenClaw.” arXiv:2604.05589, 2026. https://arxiv.org/abs/2604.05589

[12] OpenClaw Foundation. “OpenClaw.” GitHub. https://github.com/openclaw/openclaw

[13] OpenClaw. “Memory Configuration Reference.” https://docs.openclaw.ai/reference/memory-config

[14] *TechCrunch*. “OpenClaw Creator Peter Steinberger Joins OpenAI.” 15 February 2026. https://techcrunch.com/2026/02/15/openclaw-creator-peter-steinberger-joins-openai/

[15] R. Schmelzer. “OpenAI Hires OpenClaw Creator Peter Steinberger And Sets Up Foundation.” *Forbes*, 16 February 2026. https://www.forbes.com/sites/ronschmelzer/2026/02/16/openai-hires-openclaw-creator-peter-steinberger-and-sets-up-foundation/

[16] Nous Research. “Hermes Agent.” GitHub. https://github.com/NousResearch/hermes-agent

[17] Nous Research. “Architecture.” Hermes Agent Developer Guide. https://hermes-agent.nousresearch.com/docs/developer-guide/architecture

[18] Nous Research. “Persistent Memory.” Hermes Agent User Guide. https://hermes-agent.nousresearch.com/docs/user-guide/features/memory/

[19] “Memory and Sessions — NousResearch/hermes-agent.” DeepWiki. https://deepwiki.com/NousResearch/hermes-agent/4.3-memory-and-sessions

[20] Vercel. “eve — The Framework for Building Agents.” https://eve.dev/

[21] Vercel. “Execution Model and Durability.” eve Documentation. https://eve.dev/docs/concepts/execution-model-and-durability

[22] Vercel. “State.” eve Documentation. https://eve.dev/docs/guides/state

[23] AgentsORG. “Benji.” GitHub. https://github.com/AgentsORG/Benji

[24] AgentsORG. https://agents.org.in

[25] S. Khemani. “OpenPoke: Recreating Poke’s Architecture.” https://www.shloked.com/writing/openpoke

[26] *TechCrunch*. "Why Cognition Bought Poke." 24 July 2026. https://techcrunch.com/2026/07/24/why-cognition-bought-poke-ai-personality-is-becoming-a-competitive-advantage/

[27] The Pierre Computer Company. "Code Storage." https://code.storage/

[28] The Pierre Computer Company. "Code Storage Documentation." https://code.storage/docs

[29] The Pierre Computer Company. https://pierre.computer/

[30] SQLite Consortium. "Write-Ahead Logging." https://www.sqlite.org/wal.html

[31] SQLite Consortium. "FTS5 Extension." https://www.sqlite.org/fts5.html

[32] A. Kane. "pgvector." GitHub. https://github.com/pgvector/pgvector

Harshit Khemani · 241302081 · B.Tech CSE (AI/ML), Section C

Department of Computer Science & Engineering, SGT University